

Local and Global Equilibrium Effects in Discrete Graph Attention Diffusion (DGAD)

DGAD Discussion Notes

June 2026

Contents

1	Introduction	2
2	Equilibrium Effects in DGAD	3
2.1	The Core Mechanism	3
2.2	Interpretation I: Topology and Hop Count	3
2.2.1	Local Equilibrium (One-Hop Scale)	3
2.2.2	Global Equilibrium (Multi-Hop Scale)	3
2.2.3	Correspondence Between Scales	4
2.3	Interpretation II: Community Structure	4
2.3.1	Local Equilibrium Within Communities	4
2.3.2	Global Equilibrium Between Communities	4
2.3.3	Why DGAD Exhibits Both Scales	4
2.3.4	Unifying the Two Interpretations	5
3	Promoting Local Equilibrium, Slowing Global Equilibrium	6
3.1	Principle for Community Separation	6
3.2	Hyperparameter Guidance	6
3.3	Practical Regimes	7
3.4	Downstream Task Implications	7
4	Per-Edge Step Sizes τ_{ij}	8
4.1	From Scalar to Per-Edge τ	8
4.2	Design Methods	8
4.2.1	Community Priors (Labels or Detected Communities)	8
4.2.2	Structural Heuristics (No Labels)	8
4.2.3	Feature Homophily	9
4.2.4	Decoupling from Attention	9
4.2.5	Learnable τ_{ij}	9
4.2.6	Step Schedule (Orthogonal but Complementary)	9
4.3	Recommended Progression	9
4.4	Comparison with Global τ	10
5	τ_{ij} from KNN Graph Asymmetry	11
5.1	Asymmetry in Feature-Space KNN Graphs	11
5.2	Proposed Formulas	11
5.2.1	Scheme 1: Source Density	11
5.2.2	Scheme 2: Co-Density (Conservative)	11
5.2.3	Scheme 3: Density Similarity	12
5.2.4	Scheme 4: Mutual KNN + Co-Density (Recommended)	12
5.3	Integration with Diffusion	12
5.4	Qualitative Picture	12

5.5	Caveats and Limitations	12
5.6	Practical Default	12
6	Summary	14

Chapter 1

Introduction

Discrete Graph Attention Diffusion (DGAD) iteratively smooths node features on a graph by moving each node toward an attention-weighted mixture of its neighbors. A single diffusion step in `GNDLayer` takes the form

$$x_i^{(k+1)} = (1 - \tau) x_i^{(k)} + \tau \sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(k)} x_j^{(k)}, \quad (1.1)$$

where τ is the scalar `time_increment` (default 0.2), and α_{ij} are neighborhood-normalized attention weights. Stacking K such steps (`num_steps`) yields final node embeddings suitable for classification, clustering, and other downstream tasks.

A recurring phenomenon in this process is the coexistence of **local** and **global equilibrium effects** in node embeddings. This document collects two complementary interpretations of that phenomenon, discusses how to exploit the trade-off for community separation, surveys methods for designing per-edge step sizes τ_{ij} , and finally proposes a KNN-specific construction based on reverse-degree asymmetry.

Chapter 2

Equilibrium Effects in DGAD

2.1 The Core Mechanism

Each diffusion step balances two forces:

- **Retention:** the term $(1 - \tau) x_i^{(k)}$ preserves the node’s current identity;
- **Mixing:** the term $\tau \sum_j \alpha_{ij} x_j^{(k)}$ pulls the node toward its attention-weighted neighborhood aggregate.

At a *fixed point* (equilibrium) for node i , assuming $\tau > 0$,

$$x_i = \sum_{j \in \mathcal{N}(i)} \alpha_{ij} x_j. \quad (2.1)$$

The embedding equals the attention-weighted average of its neighbors. How one labels this “local” versus “global” depends on the scale at which equilibrium is observed.

2.2 Interpretation I: Topology and Hop Count

2.2.1 Local Equilibrium (One-Hop Scale)

Local equilibrium refers to the per-node, per-step attractor defined by Equation (2.1). Within a single step, each node moves toward consensus with its immediate neighbors only. The step size τ controls how far the node travels in one update:

- small τ : embeddings retain individual structure longer;
- large τ : faster convergence to the one-hop neighbor mixture.

Attention determines *which* neighbors matter and *how much*; it shapes the local attractor itself rather than applying uniform averaging.

2.2.2 Global Equilibrium (Multi-Hop Scale)

Global equilibrium emerges after K repeated local updates. Although each step mixes only one-hop neighbors, information propagates across the graph over multiple steps—analogous to the continuous-time dynamics

$$\frac{dx}{dt} = \text{Agg}(x) - x, \quad (2.2)$$

whose long-time behavior drives connected components toward graph-wide harmonization (the familiar oversmoothing limit).

Table 2.1: Two temporal–spatial scales of equilibrium.

Aspect	Local	Global
Spatial scope	1-hop neighborhood	Multi-hop / full graph
Temporal scope	Single step	Accumulated over K steps
Manifestation	Node individuality vs. neighbors	Distant nodes influence one another
Control knobs	τ (per-step mixing)	K (<code>num_steps</code>)

2.2.3 Correspondence Between Scales

Default settings ($\tau = 0.2$, $K = 8$) strike a practical balance: enough local smoothing without immediate global collapse.

2.3 Interpretation II: Community Structure

2.3.1 Local Equilibrium Within Communities

On graphs with community structure, an alternative and often more task-relevant reading applies:

- **Local equilibrium** = equilibrium *within* a community C .

Inside C , edges are dense, features are homophilous, and attention softmax is dominated by intra-community neighbors. After a few diffusion steps, nodes in C converge toward a **community-specific consensus**—similar embeddings inside C , while distinct communities remain separable. This occurs on a *short* diffusion time scale because information circulates primarily within C .

2.3.2 Global Equilibrium Between Communities

- **Global equilibrium** = equilibrium *between* communities.

Inter-community edges are sparse; bridge and boundary nodes mediate cross-community flow. Cross-community neighbors receive lower attention mass, so inter-community information exchange is slow and requires more steps. Global equilibrium means community-level representations settle into a coordinated graph-wide configuration—not necessarily identical vectors, but stable relative positions across communities.

2.3.3 Why DGAD Exhibits Both Scales

The update in Equation (1.1) naturally splits edges into two regimes:

Table 2.2: Edge regimes under community structure.

Connection	Attention pattern	Equilibration speed
Intra-community ($i, j \in C$)	Large weights, many competing neighbors	Fast $\rightarrow$ local equilibrium
Inter-community ($i \in C, j \in C'$)	Small weights, few channels	Slow $\rightarrow$ global equilibrium

Diffusion is therefore **multi-scale**:

Few steps $\Rightarrow$ communities equilibrate internally; inter-community coupling remains weak.

Many steps $\Rightarrow$ intra-community states are stable; cross-community alignment accelerates.

2.3.4 Unifying the Two Interpretations

On community-structured graphs, the hop-count and community views largely coincide: one-hop mixing predominantly occurs inside communities, while multi-hop propagation is required for sustained inter-community coupling. The community interpretation explains *why* two time scales appear—because the graph itself is heterogeneous—not merely because step count is large.

Chapter 3

Promoting Local Equilibrium, Slowing Global Equilibrium

3.1 Principle for Community Separation

To better **distinguish communities**, one should:

- **Promote local equilibrium:** encourage fast intra-community consensus (tight, coherent clusters);
- **Slow global equilibrium:** limit cross-community homogenization (preserve boundaries).

The desirable operating point is often: *intra-community equilibrium has been reached, but inter-community equilibrium has not.*

3.2 Hyperparameter Guidance

Equation (1.1) offers two primary global knobs:

Table 3.1: Global controls for the local/global trade-off.

Goal	Direction	Parameter
Promote local equilibrium	Strengthen intra-community mixing (moderately)	τ (<code>time_increment</code>)
Slow global equilibrium	Limit cross-community propagation	K (<code>num_steps</code>), or reduce τ

Caveat on τ . Very large τ accelerates *both* local and global equilibration and can cause oversmoothing. A robust strategy is:

1. **Control K first**—the most direct lever on global reach;
2. **Tune τ moderately** in combination with K ;
3. **Prefer fixed topology** (`edge_rewire=False`) when the input graph already encodes sparse cross-community links;
4. **Use distance-based attention** (`attention_type="dist"`) to up-weight similar neighbors and down-weight dissimilar ones, biasing toward “fast local, slow global” behavior when inter-community feature gaps are large.

Table 3.2: Qualitative regimes.

Setting	Outcome
Small K + moderate τ	Local equilibrium inside communities; global equilibrium delayed — favorable for
Large K + large τ	Both equilibria advance; oversmoothing — communities hard to separate

3.3 Practical Regimes

3.4 Downstream Task Implications

- **Unsupervised clustering:** the “local yes, global not yet” window often aligns with better cluster structure.
- **Node classification:** intra-community smoothing supports label consistency; preserving inter-community differences retains class boundaries.
- **Over-diffusion:** performance can degrade not from insufficient smoothing, but from *excessive global equilibrium* that erases discriminative structure.

Chapter 4

Per-Edge Step Sizes τ_{ij}

4.1 From Scalar to Per-Edge τ

The README describes a single adjustable step size τ per node update. A natural extension assigns a distinct τ_{ij} to each directed edge ($j \rightarrow i$), yielding the edge-wise incremental form

$$x_i^{(k+1)} = x_i^{(k)} + \sum_{j \in \mathcal{N}(i)} \alpha_{ij} \tau_{ij} (x_j^{(k)} - x_i^{(k)}). \quad (4.1)$$

Interpretation:

- large τ_{ij} : node i mixes rapidly along edge ($j \rightarrow i$);
- small τ_{ij} : that edge contributes little cross-node drift.

Constraint: $\tau_{ij} \in [0, 1]$. Optionally enforce a per-node budget $\sum_j \alpha_{ij} \tau_{ij} \leq \tau_{\max}$ for stability.

Implementation note. In the current `GNDLayer`, `time_increment` is applied *after* neighborhood aggregation at the node level. True per-edge τ_{ij} should enter the weighted aggregation, e.g.,

```
weighted = nodes_features_source * attentions_per_edge * tau_per_edge
```

with node-level propagate optionally reduced to a pure residual or $\tau_i \equiv 1$.

4.2 Design Methods

4.2.1 Community Priors (Labels or Detected Communities)

When communities c_i are known or estimated (Louvain, Leiden, spectral clustering):

$$\tau_{ij} = \begin{cases} \tau_{\text{in}} & \text{if } c_i = c_j, \\ \tau_{\text{out}} & \text{if } c_i \neq c_j, \end{cases} \quad \tau_{\text{in}} \gg \tau_{\text{out}}. \quad (4.2)$$

Example: $\tau_{\text{in}} \in [0.3, 0.5]$, $\tau_{\text{out}} \in [0.01, 0.05]$.

4.2.2 Structural Heuristics (No Labels)

A practical template:

$$\tau_{ij} = \tau_{\min} + (\tau_{\max} - \tau_{\min}) \cdot \sigma\left(\frac{\text{CN}_{ij} - \mu}{\sigma}\right), \quad (4.3)$$

where CN_{ij} is the common-neighbor count.

Table 4.1: Topology-based signals for τ_{ij} .

Signal	Rule of thumb	Rationale
Common neighbors / Jaccard	$\tau_{ij} \propto \text{Jaccard}(i, j)$	Shared neighborhood $\Rightarrow$ same community
Edge betweenness	$\tau_{ij} \propto 1 - \text{norm}(\text{betweenness}_{ij})$	Bridges get small τ
Effective resistance	$\tau_{ij} \propto \exp(-R_{ij}/T)$	Graphically distant pairs mix slowly
Conductance / cut edges	Small τ on cross-cut edges	Direct community-boundary control

4.2.3 Feature Homophily

Align with DGAD’s feature-space view:

$$\tau_{ij} = \tau_{\max} \cdot \exp\left(-\frac{\|x_i - x_j\|^2}{T}\right). \quad (4.4)$$

Use initial features $x^{(0)}$ for a fixed τ_{ij} , or current $x^{(k)}$ for dynamic τ . Attention α_{ij} can govern *who* to listen to; τ_{ij} governs *how fast* to diffuse along each edge.

4.2.4 Decoupling from Attention

To avoid double amplification on already-strong edges:

$$\tau_{ij} = \tau_{\text{base}} \cdot f_{ij}, \quad f_{ij} \in [0, 1], \quad (4.5)$$

where f_{ij} comes from structure or features, not directly from α_{ij} . A per-node budget normalization is also possible:

$$\tau_{ij} = \tau_{\max} \cdot \frac{f_{ij}}{\sum_{j'} \alpha_{ij'} f_{ij'}} \cdot \alpha_{ij}. \quad (4.6)$$

4.2.5 Learnable τ_{ij}

For end-to-end training:

$$\tau_{ij} = \sigma\left(w^\top [h_i \| h_j \| e_{ij}] + b\right). \quad (4.7)$$

Regularize large τ on predicted bridge edges, or distill from heuristic teachers f_{ij} .

4.2.6 Step Schedule (Orthogonal but Complementary)

Even with fixed τ_{ij} , decay over steps slows late-stage global mixing:

$$\tau_{ij}^{(k)} = \tau_{ij} \cdot \gamma^k, \quad \gamma < 1. \quad (4.8)$$

4.3 Recommended Progression

1. Fixed graph + Louvain communities: $\tau_{\text{in}} = 0.4$, $\tau_{\text{out}} = 0.02$.
2. No community labels: $\tau_{ij} \propto \text{Jaccard}(i, j)$ or $\exp(-\|x_i - x_j\|^2/T)$.
3. Joint with attention: α for weights, τ for rates; cap inter-community τ aggressively.
4. Supervised tasks: learnable τ with bridge regularization.

Monitor:

- **Local signal:** rising cosine similarity *within* communities;
- **Global signal:** *between*-community similarity stays low or rises slowly.

4.4 Comparison with Global τ

Table 4.2: Global vs. per-edge step sizes.

Method	Promote local	Slow global	Cost
Reduce K	medium	strong	risk of under-smoothing
Reduce global τ	weak	strong	slows intra-community mixing too
Per-edge τ_{ij}	strong	strong	$O(E)$ storage/compute

Per-edge τ allows large steps on intra-community edges and small steps on inter-community edges simultaneously—avoiding a single global τ that sacrifices one scale for the other.

Chapter 5

τ_{ij} from KNN Graph Asymmetry

5.1 Asymmetry in Feature-Space KNN Graphs

When building graphs via `knn_graph`, each node i selects k nearest neighbors in feature space, creating directed edges $i \rightarrow j$. In `GNDLayer`, `edge_index[0]` is the aggregation target, so node i receives messages from its chosen neighbors.

Two degree quantities differ:

Table 5.1: KNN degree asymmetry (aggregation convention).

Quantity	Meaning	On KNN graphs
In-degree (aggregation) d_i^{in}	Edges for which i is the target	Fixed $\approx k$
Reverse degree d_j^{rev}	Count of nodes that select j as a neighbor	Varies with local density

High d_j^{rev} indicates that j lies in a dense region of feature space (many points include j in their KNN list). Low d_j^{rev} suggests peripheral, isolated, or boundary locations. This asymmetry is a free structural signal for designing τ_{ij} without community labels.

5.2 Proposed Formulas

For edge $(i \leftarrow j)$ (node i receives from neighbor j):

$$\tau_{ij} = \tau_{\min} + (\tau_{\max} - \tau_{\min}) \cdot g(d_i^{\text{rev}}, d_j^{\text{rev}}). \quad (5.1)$$

5.2.1 Scheme 1: Source Density

$$g = \frac{d_j^{\text{rev}} - d_{\min}}{d_{\max} - d_{\min}}. \quad (5.2)$$

Messages from dense-region nodes diffuse faster.

5.2.2 Scheme 2: Co-Density (Conservative)

$$g = \min\left(\frac{d_i^{\text{rev}}}{d_{\max}}, \frac{d_j^{\text{rev}}}{d_{\max}}\right). \quad (5.3)$$

Large τ only when *both* endpoints are dense—boundary edges are automatically suppressed.

5.2.3 Scheme 3: Density Similarity

$$g = \exp\left(-\frac{|d_i^{\text{rev}} - d_j^{\text{rev}}|^2}{T}\right). \quad (5.4)$$

Similar reverse degrees (typical intra-community pairs) receive large τ ; mismatched pairs (dense core to sparse periphery) receive small τ .

5.2.4 Scheme 4: Mutual KNN + Co-Density (Recommended)

Cross-community edges in asymmetric KNN are often **one-way**; intra-community edges are more often **mutual**:

$$\tau_{ij} = \tau_{\max} \cdot \mathbb{1}[i \in \text{KNN}(j)] \cdot \frac{\sqrt{d_i^{\text{rev}} \cdot d_j^{\text{rev}}}}{d_{\max}}. \quad (5.5)$$

- mutual + high reverse degree $\Rightarrow$ large τ : fast **local** equilibration inside dense communities;
- non-mutual or low density $\Rightarrow$ small τ : slow **global** coupling.

5.3 Integration with Diffusion

Substitute τ_{ij} into Equation (4.1). Precompute d^{rev} from `edge_index` in $O(|E|)$ once per graph build. When `edge_rewire=True`, recompute after each KNN reconstruction.

5.4 Qualitative Picture

Dense community interior: high $d^{\text{rev}} \Rightarrow$ large $\tau \Rightarrow$ fast local equilibrium.

Boundaries and bridges: low or asymmetric $d^{\text{rev}} \Rightarrow$ small $\tau \Rightarrow$ delayed global equilibrium.

Per-edge diffusion rates follow the geometric role of each KNN edge rather than a single global step size.

5.5 Caveats and Limitations

1. **Dense–dense bridges**: two community cores can both have high d^{rev} . Schemes 1–2 alone may assign large τ to inter-community bridges; mutual-KNN (Scheme 4) or density-similarity (Scheme 3) mitigates this.
2. **Dependence on k** : larger KNN k shifts all d^{rev} upward; prefer quantile or $d_{\max}$ normalization over raw counts.
3. **Dynamic rewiring**: d^{rev} changes when KNN is rebuilt each step; refresh τ_{ij} accordingly.
4. **Fixed non-KNN graphs** (e.g., citation networks): reverse-degree asymmetry is absent; use Jaccard, community detection, or feature distance instead.

5.6 Practical Default

A strong starting formula:

$$\tau_{ij} = \tau_{\max} \cdot \mathbb{1}[i \in \text{KNN}(j)] \cdot \exp\left(-\frac{|d_i^{\text{rev}} - d_j^{\text{rev}}|^2}{T}\right), \quad (5.6)$$

with $\tau_{\max} \in [0.3, 0.5]$ and T set to one or two times the variance of $\{d_i^{\text{rev}}\}$. Success is indicated when intra-community embedding similarity rises quickly while inter-community similarity remains comparatively low.

Chapter 6

Summary

DGAD diffusion produces **local** and **global equilibrium effects** at two complementary levels:

1. **Hop scale:** one-hop neighbor mixing vs. multi-hop graph-wide harmonization;
2. **Community scale:** fast intra-community consensus vs. slow inter-community coupling on structured graphs.

For community separation, favor settings where local equilibria have formed inside communities but global equilibration across communities is still incomplete. Global knobs (K, τ) provide coarse control; **per-edge** τ_{ij} provides fine control—assigning large rates to homophilous, dense, mutual-KNN edges and small rates to sparse, asymmetric, or bridge-like edges.

On KNN graphs, fixed in-degree and variable reverse degree offer a particularly natural, label-free signal: *popularity as a neighbor* proxies local feature density and can be turned directly into a heterogeneous diffusion schedule that promotes local equilibrium while retarding global equilibrium.